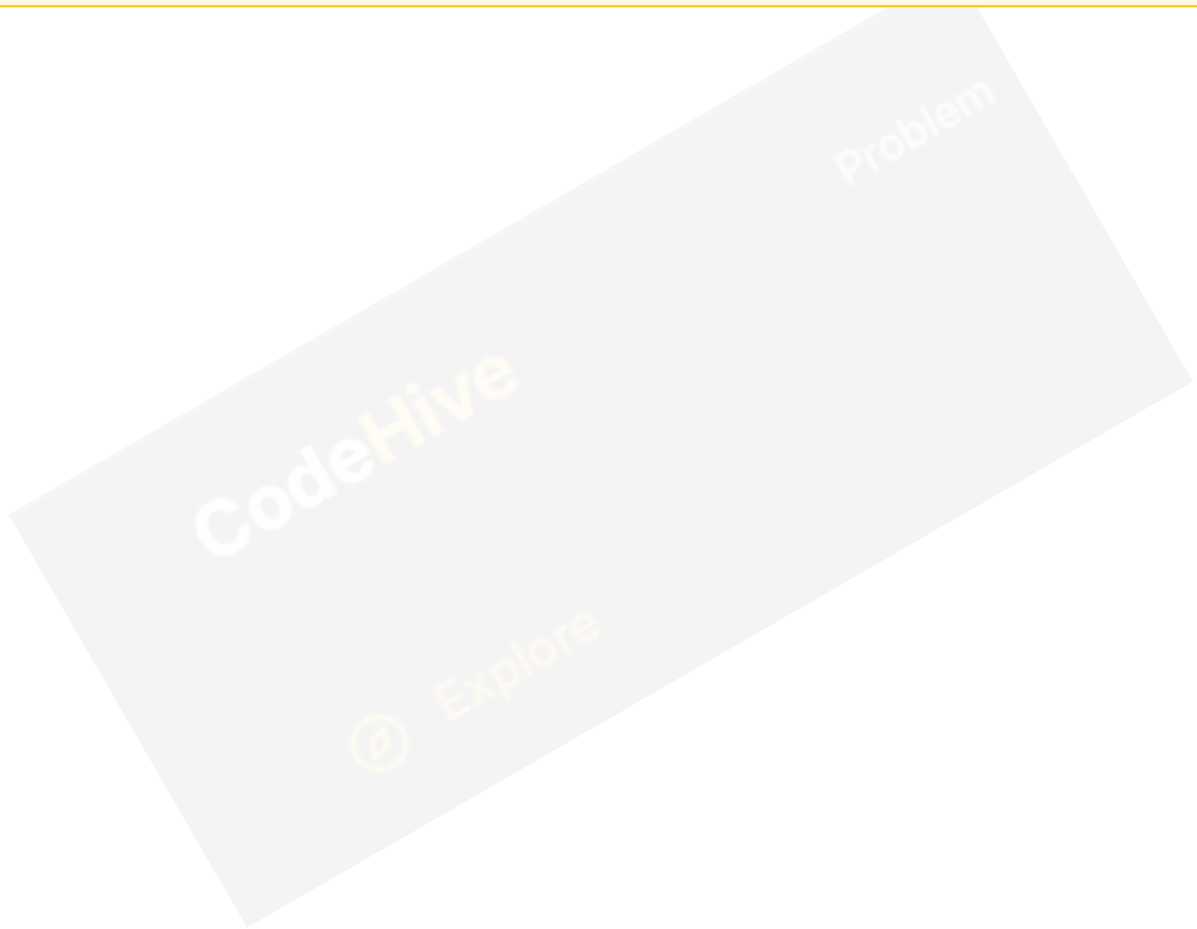


Structure Padding in C

Structure padding is the hidden mechanism where the compiler inserts unused bytes between structure members to ensure proper **Memory Alignment**.

One-Line Definition: *Structure padding is the insertion of unused bytes by the compiler to align structure members in memory for faster CPU access.*



1. Why Padding is Needed?

Modern CPUs do not read memory byte-by-byte. Instead, they read in "words" (usually 4 or 8 bytes at a time).

- **Performance:** Accessing unaligned data requires the CPU to do two memory reads instead of one.
- **Hardware Requirement:** Certain architectures will actually crash if you try to read unaligned data.



2. Memory Alignment Rules

Most compilers follow the **Self-Alignment** rule: a data member must start at a memory address that is a multiple of its own size.

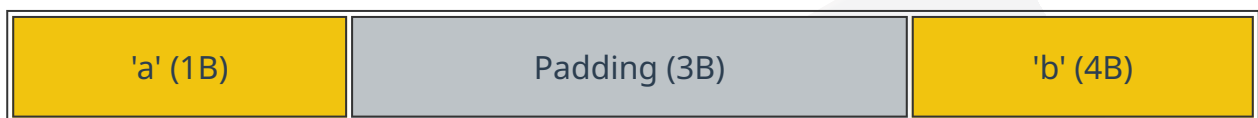
- `char` (1 byte): Can start at any address.
- `int` (4 bytes): Must start at an address divisible by 4.
- `double` (8 bytes): Must start at an address divisible by 8.



3. Working Example

```
struct Test {  
    char a;    // 1 byte  
    int b;     // 4 bytes  
};
```

Visual Memory Layout:



Total size = **8 bytes** (despite contents being 5 bytes).

4. Larger Padding Example

```
struct Example {  
    char c;  
    double d;  
};
```

Here, reaching the `double` requires alignment to 8 bytes. The compiler adds 7 bytes of padding after the character `c`.

Total Size = **16 bytes**.

5. Reducing Memory Wastage

You can optimize memory usage by **reordering** your structure members from **largest to smallest** data type.

```
// Optimized Version
struct Optimized {
    double d; // 8B
    int i;    // 4B
    char c;   // 1B
};
```

This minimizes the gaps the compiler needs to fill.

CodeHive

@ Explore

6. Structure Packing

If memory is more important than speed (e.g., sending data over a network), you can tell the compiler NOT to add padding using `#pragma pack(1)`.

```
#pragma pack(1)
struct Packed {
    char a;
    int b;
}; // Size: 5 bytes
```

Warning: This makes data access slower.

CodeHive

@ Explore

7. System Architecture dependency

Padding behavior differs between:

- **32-bit Systems:** Usually aligns to 4-byte boundaries.
- **64-bit Systems:** Usually aligns to 8-byte boundaries.

Always use `sizeof()` to determine the actual size during runtime.



8. Why Developers should care? ★

In high-performance applications like game engines or real-time trading systems, "Cache Misses" caused by poor alignment can slow down the software by 10x.

Understanding padding allows you to write **Professional Grade C Code**.



Key Points (Exam Ready)

- Padding is automatic and system-dependent.
- It trades **Memory** for **Speed**.
- Avoid wastage by placing larger types first.
- Use `sizeof ( )` to verify padding in your compiler.

CodeHive Academy - Tech Insights

CodeHive

@ Explore